Proyecto Aplicado – Ingeniería de Software con IA

INFORME TÉCNICO

Proyecto Aplicado – Ingeniería de Software con IA

Estudiante: Victor manuel Cañas Salazar

Programa / institución: ingenieria de software

Fecha: 28 de septiembre de 2026

Documento técnico de integración de Claude Code, Claude API y Model Context

Protocol (MCP).
Proyecto Aplicado – Ingeniería de Software con IA

ÍNDICE

1. Introducción

2. Objetivos

3. Metodología y alcance

4. Parte A – Formación y conceptos adquiridos

5. Parte B – Flujo de trabajo con Claude Code

6. Parte C – Aplicación Python con Claude API

7. Parte D – Servidor propio con MCP

8. Arquitectura general del proyecto

9. Evaluación, pruebas y resultados

10. Dificultades y decisiones técnicas

11. Lecciones aprendidas

12. Preguntas conceptuales

13. Conclusiones

14. Estado final y evidencias pendientes

1. Introducción

Este proyecto aplicado integra conceptos de ingeniería de software con capacidades de inteligencia artificial generativa. El trabajo se organiza alrededor de tres tecnologías principales: Claude Code para apoyar el trabajo sobre un repositorio existente, Claude API para construir una aplicación programática y Model Context Protocol (MCP) para exponer herramientas, recursos y prompts mediante un protocolo estandarizado.

El objetivo no es únicamente utilizar una herramienta de inteligencia artificial, sino documentar un proceso de ingeniería: comprender un sistema existente, definir una funcionalidad, planificar cambios, implementar código, realizar pruebas, registrar decisiones y construir componentes que puedan ser evaluados de forma objetiva.

Durante el desarrollo surgió una limitación de acceso: no se contó con una clave institucional para la API de Claude ni con acceso autorizado a Claude Code. Por esta razón, el informe diferencia entre actividades realmente ejecutadas, preparación técnica realizada y actividades que quedan pendientes de evidencia. No se presentan como reales llamadas, resultados, capturas o interacciones que no hayan sido ejecutadas.

2. Objetivos

Objetivo general: desarrollar un proyecto de ingeniería de software que demuestre comprensión práctica de Claude Code, Claude API y MCP, manteniendo trazabilidad, pruebas y documentación.

Objetivos específicos: comprender los conceptos fundamentales de los cuatro cursos de formación; documentar

un flujo Explore
→
Plan
→
Program
→
Commit; construir una base Python preparada para una aplicación con Claude API; diseñar un servidor MCP con tres herramientas, un recurso y un prompt; establecer una estrategia de evaluación con casos reproducibles; y consolidar los resultados en un repositorio público con documentación técnica.

3. Metodología y alcance

La metodología seguida combina documentación, implementación incremental y validación. Primero se revisó la estructura del proyecto base y se preparó el repositorio. Después se definieron convenciones mediante CLAUDE.md y se registró la trazabilidad en bitacora.md. Se diseñó una funcionalidad académica, se implementó con TypeScript y se añadieron pruebas. Paralelamente se prepararon las Partes C y D.

El alcance se limita a un sistema académico de ejemplo. Las calificaciones utilizadas son datos simulados y no representan información académica real. La aplicación no debe utilizarse para manejar datos personales reales sin controles adicionales de autenticación, autorización, almacenamiento seguro y auditoría.

Parte A – Formación y conceptos adquiridos

Claude Code 101

Los conceptos principales trabajados fueron el contexto del proyecto, la comprensión del código existente y el uso controlado de instrucciones. La idea central es que una herramienta de programación asistida necesita conocer la estructura, convenciones y restricciones del repositorio para proponer cambios coherentes.

Aplicación al proyecto: se creó CLAUDE.md para documentar el propósito del sistema, tecnologías, estructura, convenciones, comandos de prueba y restricciones. Esto deja una referencia persistente para cualquier agente o desarrollador que trabaje posteriormente sobre el repositorio.

Claude Code in Action

Se estudiaron exploración, planificación, revisión y ejecución de cambios. La práctica propuesta se organiza en

Explorar
→
Planificar
→
Programar
→
Probar
→
Revisar
→
Commit. La bitácora permite registrar propuestas aceptadas o rechazadas y justificar las decisiones.

Aplicación: se documentó un plan para incorporar un resumen académico y se realizaron cambios pequeños con commits descriptivos. Debido a la falta de acceso

autorizado a Claude Code, la preparación y programación de esta parte fueron realizadas manualmente y se identifican como tales.

Building with the Claude API

Los conceptos principales son integración mediante API, system prompt, historial de conversación, herramientas, respuestas estructuradas y streaming. La aplicación debe separar instrucciones del sistema de las entradas del usuario y validar las respuestas antes de utilizarlas.

Aplicación: la carpeta parte-c-api contiene un prototipo local con system prompt, historial, esquema de respuesta JSON, dos herramientas definidas por el estudiante y validación básica. La llamada real a Claude queda pendiente por falta de API key.

Introduction to Model Context Protocol

MCP proporciona una forma estandarizada de exponer capacidades a un cliente mediante herramientas, recursos y prompts. En el proyecto se implementó un servidor académico con tres herramientas, un recurso y un prompt.

Aplicación: parte-d-mcp/server.py contiene consultar_calificacion, calcular_promedio y validar_estudiante; el recurso academico://reglas; y el prompt analizar_rendimiento.

Parte B – Flujo de trabajo con Claude Code

Explorar

La primera actividad consiste en entender el repositorio antes de modificarlo. Se identificaron archivos TypeScript, configuración de TypeScript, frontend estático y dependencias Node. También se identificó una función existente para consultar calificaciones y un flujo HTTP /chat.

Planificar

Se creó plan.md con una propuesta para una funcionalidad de resumen académico. La funcionalidad debía reutilizar la información existente, calcular el promedio, validar el caso de estudiante inexistente y mantenerse separada de la lógica HTTP.

Programar

Se creó src/academico.ts con obtenerResumenAcademico y se agregaron pruebas en tests/academico.test.ts. La función devuelve materias y promedio, maneja estudiantes inexistentes y contempla el caso sin calificaciones.

Probar

El comando npm test fue configurado para ejecutar las pruebas mediante tsx. La ejecución local reportó que las pruebas del resumen académico pasaron correctamente. Esta prueba corresponde a la implementación local y no a una ejecución realizada por Claude Code.

Commit y trazabilidad

El repositorio utiliza commits descriptivos. Entre los cambios documentales realizados se encuentran la preparación de la documentación de Claude Code y el plan de la nueva funcionalidad. El historial debe conservarse como evidencia de evolución del trabajo.

Limitación

La parte que requiere demostrar una interacción real con Claude Code no debe presentarse como realizada. El repositorio contiene la preparación necesaria, pero la evidencia del agente debe añadirse únicamente si posteriormente se dispone de acceso autorizado.

Parte C – Aplicación Python con Claude API

Diseño funcional

Se diseñó un asistente para apoyar a un estudiante de Ingeniería de Software. El system prompt establece que el asistente debe ayudar con programación y software, explicar errores, proponer soluciones razonadas, proteger información sensible y respetar formatos estructurados.

Historial multi-turno

La clase AsistenteAcademico mantiene una lista de mensajes con roles user y assistant. El método agregar_mensaje incorpora nuevos turnos y obtener_historial devuelve una copia para evitar modificar directamente la estructura interna.

Respuesta estructurada

La función respuesta_json genera un objeto con tres campos obligatorios: concepto, explicacion y recomendacion. La validación comprueba que el conjunto de claves coincida con el esquema esperado. En una integración real, esta validación debe complementarse con la validación de la respuesta recibida de la API.

Herramientas definidas por el estudiante

Se definieron herramienta_buscar_archivo y herramienta_resumir_codigo. La primera valida un nombre de archivo y prepara una consulta local; la segunda valida la existencia de código y devuelve información básica para su análisis. El ciclo completo de tool call de Claude requiere una API key y una integración real; por ello no se presenta como ejecutado.

Streaming y errores

El requisito académico contempla streaming y manejo de errores de API, red y respuestas inválidas. La versión preparada deja identificada esta necesidad, pero no se debe afirmar que el streaming real de Claude o el

manejo de errores de red de la API fueron validados mientras no exista una llamada real.

Seguridad de la API key

Se creó .env.example con ANTHROPIC_API_KEY como variable de configuración. La clave real no debe incluirse en Git, README, capturas ni archivos del repositorio. El .gitignore excluye archivos .env y conserva únicamente .env.example.

7. Parte D – Servidor propio con MCP

Componentes

El servidor utiliza FastMCP del SDK de Python. Incluye tres herramientas: consultar_calificacion, calcular_promedio y validar_estudiante. También incluye el recurso academico://reglas y el prompt analizar_rendimiento.

Herramienta consultar_calificacion

Recibe estudiante_id y materia y consulta una base de datos simulada en memoria. Si el estudiante o la materia no existen, devuelve un mensaje controlado. Los datos son únicamente de demostración.

Herramienta calcular_promedio

Recibe una lista de notas y devuelve el promedio redondeado a dos decimales. Para una lista vacía devuelve 0.0, evitando una división por cero.

Herramienta validar_estudiante

Comprueba la existencia de un identificador dentro del conjunto de estudiantes de ejemplo y devuelve un objeto con estado, identificador y mensaje.

Recurso y prompt

El recurso academico://reglas contiene reglas generales sobre privacidad y manejo de calificaciones. El prompt analizar_rendimiento orienta un análisis de materias, notas y promedio sin inventar información.

Validación

La compilación sintáctica de server.py fue ejecutada mediante py_compile sin mostrar errores. La prueba con MCP Inspector debe documentarse con capturas reales de las herramientas, el recurso y el prompt. Estas evidencias se incorporarán cuando finalice la ejecución del Inspector.

8. Arquitectura general del proyecto

La arquitectura se puede dividir en cuatro capas de trabajo. La primera es el repositorio académico existente, que contiene el backend TypeScript y la interfaz web. La segunda corresponde a la Parte B, donde se documenta el proceso de ingeniería asistida. La tercera es la aplicación Python de la Parte C, diseñada para interactuar con Claude API. La cuarta es el servidor MCP de la Parte D, que expone capacidades académicas estandarizadas.

Flujo conceptual: Usuario
→
interfaz o cliente
→
lógica de aplicación
→
herramienta/recurso
→
resultado

validado. En el caso de Claude API, el modelo puede recibir un mensaje, decidir si necesita una herramienta, recibir el resultado de esa herramienta y continuar la respuesta. En MCP, un cliente compatible descubre las herramientas, recursos y prompts publicados por el servidor.

Separación de responsabilidades: TypeScript gestiona el sistema académico original; Python concentra la integración propuesta con Claude API; MCP concentra la exposición de capacidades externas; documentación y evidencias permiten verificar cada etapa.

Componente	Responsabilidad
Proyecto base	Servidor web, datos académicos simulados e interfaz.
Parte B	Proceso de exploración, planificación, implementación, pruebas y commits.
Parte C	Asistente Python, system prompt, historial, JSON y herramientas.
Parte D	Servidor MCP con 3 herramientas, 1 recurso y 1 prompt.
Evidencias	Capturas, resultados de pruebas y trazabilidad.

9. Evaluación, pruebas y resultados

La evaluación de la Parte C fue diseñada con diez casos: explicación de función, explicación de clase, comparación, excepciones, REST API, análisis de código, respuesta JSON, contexto multi-turno, búsqueda de archivo y resumen de código. Estos casos sirven como conjunto de evaluación, pero no deben presentarse como resultados de Claude si no fueron ejecutados mediante la API.

En el proyecto local se validó el cálculo del resumen académico mediante pruebas de TypeScript. Los escenarios cubren un estudiante existente, tres materias, promedio 4.7, estudiante inexistente y otro estudiante con promedio 4.35.

Para MCP se realizó una comprobación sintáctica de server.py. La siguiente etapa de validación consiste en ejecutar MCP Inspector y probar cada herramienta con entradas válidas y, cuando sea pertinente, entradas límite. Se deben conservar capturas reales como evidencia.

Criterios de aceptación propuestos: el código debe compilar; las pruebas automatizadas deben pasar; las respuestas estructuradas deben contener los campos requeridos; las herramientas deben rechazar entradas vacías o inexistentes de forma controlada; las claves no deben quedar en el repositorio; y MCP debe exponer exactamente las capacidades documentadas.

10. Dificultades y decisiones técnicas

La principal dificultad fue la disponibilidad de acceso a Claude Code y a la API. En lugar de simular resultados, se decidió preparar toda la estructura que podía validarse localmente y dejar identificadas las actividades que requieren acceso externo.

Otra dificultad fue la instalación del ecosistema MCP. La instalación inicial encontró un tiempo de espera al descargar paquetes desde Python Package Index. Posteriormente se ajustó la dependencia del proyecto para utilizar la línea de versiones compatible con la implementación FastMCP utilizada por server.py.

También se decidió conservar el proyecto original sin sustituir index.ts. La nueva lógica académica se aisló en src/academico.ts para facilitar pruebas y reducir el acoplamiento con Express.

11. Lecciones aprendidas

Una primera lección es que la IA debe incorporarse a un proceso de ingeniería y no reemplazar la revisión humana. CLAUDE.md ayuda a establecer contexto y restricciones, pero el desarrollador sigue siendo responsable de revisar el plan y los cambios.

Una segunda lección es la importancia de validar las salidas de un modelo. Un JSON con apariencia correcta no garantiza que tenga los campos esperados ni que los valores sean válidos para el sistema.

Una tercera lección es que herramientas, recursos y prompts cumplen funciones diferentes en MCP. Las herramientas representan acciones invocables, los recursos aportan información identificable y los prompts proporcionan plantillas o instrucciones reutilizables.

Finalmente, la trazabilidad es parte del producto. Los commits, bitácora, pruebas y capturas permiten distinguir una implementación real de una afirmación no verificable.

12. Preguntas conceptuales

1. ¿Qué es una ventana de contexto? Es la cantidad de información que un modelo puede considerar dentro de una interacción. Incluye instrucciones, mensajes y otros contenidos que el sistema incorpore al contexto. Una ventana limitada obliga a administrar historial y seleccionar información relevante.

2. ¿Qué son los subagentes? Son procesos o agentes especializados que pueden encargarse de una parte de una tarea y devolver resultados al flujo principal. Su utilidad depende de una división clara del trabajo y de mecanismos para revisar sus resultados.

3. ¿Para qué sirve CLAUDE.md? Sirve como documentación persistente del proyecto para indicar contexto, convenciones, comandos, restricciones y criterios de trabajo que deben respetarse al utilizar Claude Code.

4. ¿Cuál es la diferencia entre system prompt y user prompt? El system prompt define reglas y comportamiento de mayor nivel para el asistente. El user prompt contiene la solicitud concreta de la persona. Separarlos ayuda a controlar responsabilidades e instrucciones.

5. ¿Cuál es el ciclo de una herramienta en una API? El modelo recibe contexto, determina que necesita una herramienta, produce una solicitud estructurada de

herramienta, la aplicación ejecuta la función, devuelve el resultado al modelo y el modelo genera o continúa la respuesta final.

6. ¿Qué es MCP? Es un protocolo para conectar clientes de IA con servidores que ofrecen herramientas, recursos y prompts de una manera estandarizada.

7. ¿Qué diferencia existe entre tools, resources y prompts en MCP? Una tool representa una capacidad accionable; un resource representa información que puede ser consultada; y un prompt es una plantilla de instrucciones reutilizable.

8. ¿Qué es prompt injection? Es un intento de introducir instrucciones dentro de la entrada o contenido procesado para modificar el comportamiento previsto del sistema. La defensa requiere separar instrucciones confiables de datos, limitar herramientas, validar entradas y aplicar controles de autorización.

9. ¿Cómo se realiza una evaluación objetiva de un LLM? Mediante casos definidos previamente, criterios verificables y resultados observables. Es importante separar la evaluación de una impresión subjetiva y registrar errores y casos límite.

10. ¿Cuáles son limitaciones importantes de los LLM? Pueden producir información incorrecta, interpretar de forma errónea instrucciones, ser sensibles al contexto disponible y generar respuestas plausibles sin garantía de verdad. Por eso se necesitan pruebas, validación y supervisión humana.

13. Conclusiones

El proyecto permitió estructurar una experiencia práctica alrededor de tres componentes de IA aplicada al desarrollo: asistencia sobre código, consumo programático de un modelo y exposición de capacidades mediante MCP. La organización del repositorio, la separación de responsabilidades y las pruebas locales constituyen una base técnica verificable.

El resultado también demuestra la importancia de declarar las limitaciones del entorno. La falta de acceso a Claude Code y a una API key impide afirmar que determinadas integraciones externas fueron ejecutadas. En consecuencia, el proyecto diferencia preparación, ejecución local y evidencia pendiente.

La experiencia muestra que una solución con IA debe evaluarse como cualquier otro componente de software: con requisitos claros, pruebas, manejo de errores, seguridad, documentación y trazabilidad.

14. Estado final y evidencias pendientes

Estado preparado: documentación de formación, CLAUDE.md, bitácora, plan, funcionalidad académica TypeScript, pruebas locales, aplicación Python base para Claude API, servidor MCP, requirements.txt y README de componentes.

Evidencia pendiente: capturas de MCP Inspector y cualquier demostración de conexión con Claude Desktop o Claude Code que pueda ejecutarse legítimamente. También queda pendiente la ejecución real de la API de Claude si se obtiene una API key autorizada.

Regla de integridad académica: las capturas y resultados deben corresponder a ejecuciones reales. Si una actividad no puede ejecutarse por falta de acceso, debe indicarse como pendiente y conservarse la evidencia de la limitación.